

Week 4 Submission

Complex Data Handling: File Uploads & Relational Data

Track	Duration	Format	Stack
Full Stack	Week 4 of 4 (Final)	Practical Build	React + Express + MongoDB + Multer

Task Summary

This final submission extends the Week 1-3 Inventory Management module with product image uploads and a genuine relational link between inventory items and a new Supplier collection. Images are validated and previewed client-side, uploaded as multipart/form-data, stored on the server, and linked to their item record; each item can optionally reference a supplier by id, resolved into full supplier details via Mongoose population rather than duplicated data.

Repository

github.com/Rahmatsoga/week1-task → Week4_Task/

1. What Was Implemented

- Added a Supplier collection (name, contactEmail, phone) as a genuinely separate entity from Item.
- Item now references a Supplier by ObjectId, resolved via `.populate()` rather than duplicating supplier details onto every item — a real one-to-many relationship.
- POST/PUT `/api/items` now accept multipart/form-data: text fields plus an optional image file, handled server-side by Multer.
- Server-side validation of uploaded files: allowed MIME types (JPEG/PNG/WEBP/GIF) and a 5MB size limit, both enforced before the file is written to disk.
- Client-side validation mirrors the server: file type and size are checked immediately on selection, before any request is sent.
- Live image preview using `URL.createObjectURL`, shown before the item is ever submitted.
- Uploaded images are served as static files and automatically deleted from disk when an item's image is replaced or the item itself is deleted.
- New Supplier endpoints (GET/POST `/api/suppliers`, DELETE `/api/suppliers/:id`), with delete protection if a supplier is still referenced by any item.

Evaluation Criteria Coverage

Criterion	Marks	Status
File Upload	6	Complete
Validation	5	Complete
DB Relations	5	Complete
Image Preview	4	Complete
Error Handling	5	Complete

2. How It Was Implemented

2.1 File Upload Pipeline

The frontend builds a `FormData` object (`client/src/services/itemService.js`) — regular fields via `formData.append()`, the file via `formData.append("image", file)`. Axios sends this as multipart/form-data. On the server, Multer (`middleware/upload.js`) intercepts the request before the controller runs: it validates the file's MIME type and size, writes it to `server/uploads/` with a collision-proof filename (timestamp + random suffix), and populates `req.file`. The controller saves only the relative path (e.g. `"/uploads/167213-482.jpg"`) on the Item document, and `server.js` serves that folder statically so the path is directly loadable as an image URL.

2.2 Why Store a File Path, Not the File Itself, in MongoDB

Storing large binary data directly inside MongoDB documents bloats the database and slows down queries that don't even need the image. Instead, the actual file lives on disk, and MongoDB stores only a small text pointer to it — the same "reference, don't duplicate" principle applied to the Supplier relationship below.

2.3 Validation on Both Ends

File type and size are checked twice: immediately in the browser on file selection (fast feedback, no wasted upload), and again on the server via Multer's `fileFilter` and `limits` (the only check that can actually be trusted, since a client-side check alone could be bypassed). Rejected uploads return a 400 with a specific message, surfaced directly in the form.

2.4 Relational Data via Supplier References

`Item.supplier` stores a `mongoose.Schema.Types.ObjectId` referencing a `Supplier` document, rather than copying that supplier's name/email/phone directly onto every item. `GET /api/items` resolves this reference via `.populate("supplier")`, returning the full current supplier details alongside each item. Updating a supplier's details once automatically reflects across every item that references them, and deleting a supplier still in use is blocked with a 409 response rather than leaving items pointing at a deleted record.

2.5 Client-Side Preview

`URL.createObjectURL` creates a temporary, local-only link to the file still sitting on the user's device — nothing has been uploaded anywhere at this point. This lets the form show a live thumbnail immediately after file selection, and the object URL is explicitly revoked (`URL.revokeObjectURL`) when replaced or the component unmounts, to avoid leaking browser memory.

3. Demo Walkthrough & Evidence

The steps below verify the reviewer checkpoint for Week 4: "Reviewer can upload an image, see preview, submit, and reload the persisted record."

3.1 Backend running with suppliers and items seeded

```
MongoDB connected: 127.0.0.1/inventory_db
Seeded 3 suppliers and 26 inventory items.
PS C:\Users\Rahmatullah\Downloads\week1 task\Week4 Task\server> npm run dev

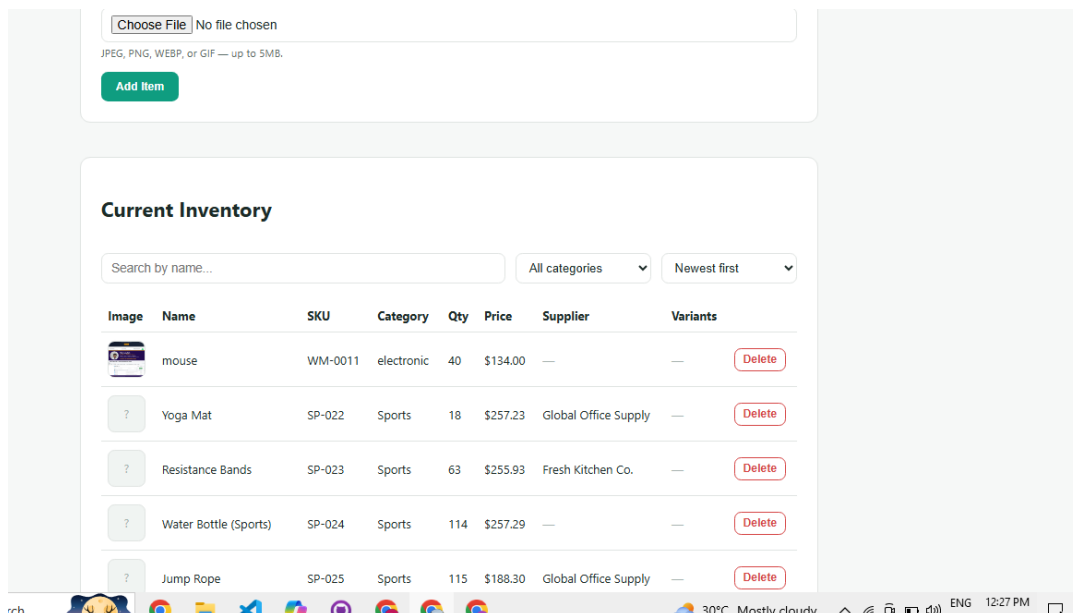
> inventory-server@4.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on http://localhost:5000
MongoDB connected: 127.0.0.1/inventory_db
```

Server running, MongoDB connected, and the seed script confirming 3 suppliers and 26 items.

3.2 Uploaded image and supplier relationship displayed correctly

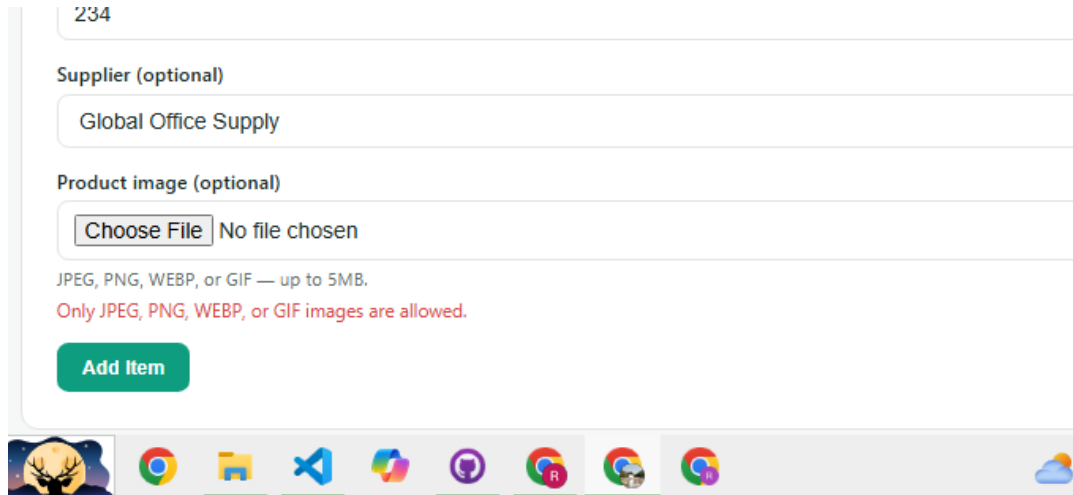
After uploading an image and linking a supplier, both appear correctly in the inventory table: a real thumbnail (not a placeholder) for the uploaded item, and resolved supplier names ("Global Office Supply," "Fresh Kitchen Co.") for items that reference one. Items without an image or supplier correctly show a placeholder and a dash, rather than an error:



Real image thumbnail and populated supplier names showing correctly in the inventory table.

3.3 Invalid file type rejected

Attempting to select a non-image file (a PDF) is rejected immediately, client-side, with a clear message before any request is sent to the server:



The screenshot shows a web form with the following fields:

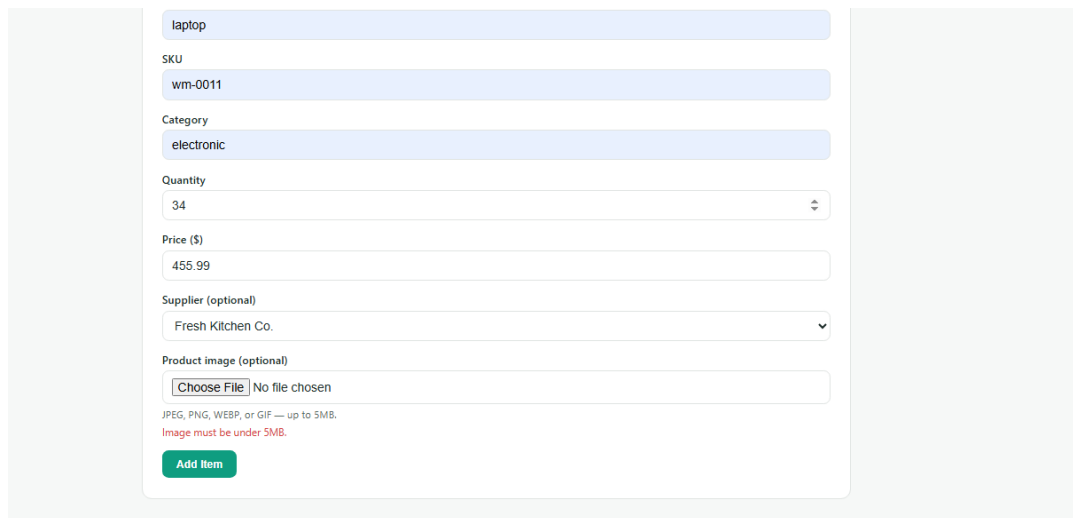
- Item ID: 234
- Supplier (optional): Global Office Supply
- Product image (optional): Choose File No file chosen

Below the product image field, there is a message: "JPEG, PNG, WEBP, or GIF — up to 5MB. Only JPEG, PNG, WEBP, or GIF images are allowed." A green "Add Item" button is at the bottom of the form. The browser's taskbar is visible at the bottom, showing various application icons.

"Only JPEG, PNG, WEBP, or GIF images are allowed" — non-image file correctly rejected.

3.4 Oversized file rejected

Selecting an image over the 5MB limit is also rejected client-side, with a specific, clear error message:



The screenshot shows a web form with the following fields:

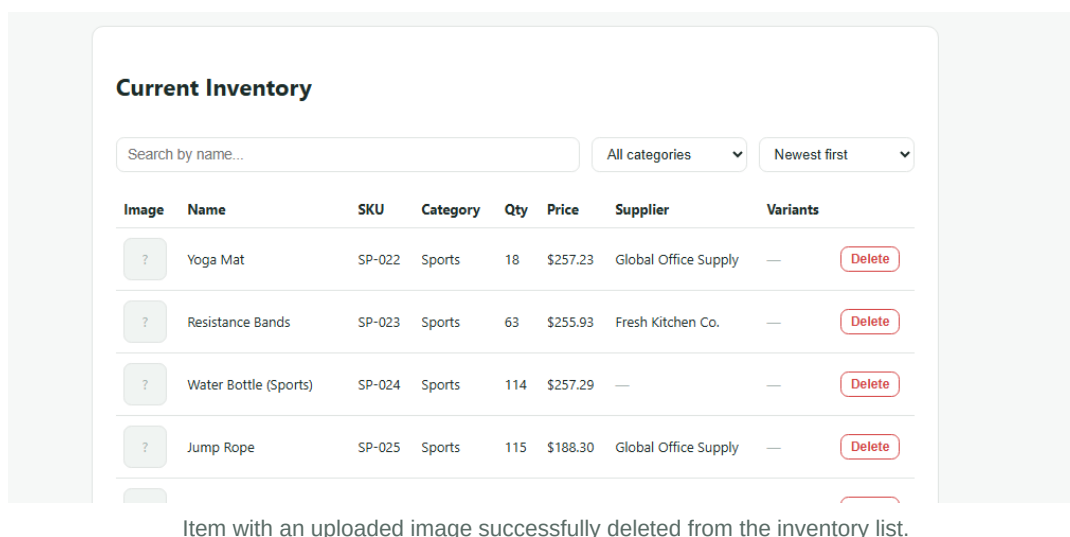
- Item Name: laptop
- SKU: wm-0011
- Category: electronic
- Quantity: 34
- Price (\$): 455.99
- Supplier (optional): Fresh Kitchen Co.
- Product image (optional): Choose File No file chosen

Below the product image field, there is a message: "JPEG, PNG, WEBP, or GIF — up to 5MB. Image must be under 5MB." A green "Add Item" button is at the bottom of the form.

"Image must be under 5MB" — oversized file correctly rejected.

3.5 Delete removes the item and its uploaded image

Deleting an item that had an uploaded image removes it from the inventory list entirely. This also confirms the backend's cleanup logic (`deleteImageFile`) ran, removing the corresponding file from `server/uploads/` on disk rather than leaving an orphaned file behind:



3.6 Verified behavior

- Selecting a valid image shows a live client-side preview before submission.
- Selecting a non-image file is rejected client-side with a clear error, before any request is sent.
- Selecting an image over 5MB is rejected client-side with a clear error.
- Submitting a valid item with an image and a linked supplier succeeds, and both appear correctly in the table.
- Items without an image or supplier display the correct fallback (placeholder / dash) rather than an error.
- Refreshing the browser preserves the uploaded image and supplier link, confirming persistence in MongoDB and on disk rather than local state.
- Deleting an item removes both the database record and its uploaded image file.

4. Tradeoffs & Known Limitations

- Files are stored on the local server disk, not cloud object storage (e.g. S3) — fine for this evaluation environment, but wouldn't scale across multiple server instances in a real production deployment.
- Server-side validation checks the browser-reported MIME type, not the file's actual binary signature ("magic bytes") — a reasonable safeguard for this project, though a production system handling untrusted uploads at scale would typically add that deeper check too.
- The create/edit form doesn't yet expose variant management — that remains a create-via-API-only feature, consistent with Week 3's scope.
- No image resizing/compression on upload; images are stored as-is up to the 5MB limit.

5. Improvements With More Time

- Move file storage to a cloud provider (e.g. AWS S3) behind a signed-URL pattern.
- Add server-side image resizing/thumbnail generation on upload (e.g. via sharp).
- Add a dedicated supplier management UI (currently suppliers can be created via the API but there's no dedicated frontend page for it).
- Verify uploaded files' actual binary signature server-side, not just the reported MIME type, for stronger validation against spoofed file types.